

Semana 1 - Backend

Documentação do Backend — Semana 1

O que foi feito

Nesta semana foi realizada a estruturação inicial do backend. O foco foi configurar o ambiente, ativar as dependências e criar os primeiros endpoints da API REST, ainda sem integração com o banco de dados.

Arquivos criados ou alterados

`app/app.py`

Arquivo principal da aplicação Flask. Contém todas as rotas da API nesta etapa.

Dependências importadas:

- `Flask` — framework principal
- `flask_cors` — habilita CORS para todas as rotas (necessário para o frontend consumir a API)
- `flask_sqlalchemy` — ORM para futura integração com o PostgreSQL
- `python-dotenv` — carrega variáveis de ambiente do arquivo `.env`

Configurações aplicadas:

- CORS habilitado globalmente
 - Conexão com o banco configurada via variável de ambiente `DATABASE_URL`
 - SQLAlchemy inicializado (sem modelos ainda — integração com banco será feita na Semana 2)
 - Variáveis de ambiente carregadas com `load_dotenv()` no início do arquivo
-

Endpoints implementados

`GET /api/status`

Retorna o status da API.

Resposta:

```
1 {  
2   "status": "online",  
3   "service": "API Monitoramento",
```

```
4  "version": "1.0.0"
5  }
6
```

Código HTTP: 200 OK

GET /health

Endpoint de verificação de saúde simples. Usado pelo Docker e por serviços externos para checar se a API está no ar.

Resposta:

```
1  {"status": "ok"}
2
```

Código HTTP: 200 OK

GET /api/hello

Rota de verificação básica criada na inicialização do projeto. Mantida para referência.

Resposta:

```
1  {"message": "Olá do BackEnd Flask!"}
2
```

Código HTTP: 200 OK

POST /api/discovery

Recebe o payload de hardware enviado pelo agente coletor. Por enquanto apenas registra os dados no log do servidor — a persistência no banco será implementada na Semana 2.

O endpoint identifica automaticamente se o host é virtualizado ou físico lendo o campo `environment.is_virtualized` do JSON recebido, sem necessidade de dois endpoints separados.

Payload esperado (enviado pelo agente):

```
1  {
2    "metadata": {
3      "schema_version": "1.0",
4      "notes": [...]
5    },
6    "environment": {
7      "is_virtualized": true,
8      "hypervisor": "VMware",
9      "source": "lscpu"
```

```

10 },
11 "cpu": {
12     "model_name": "AMD Ryzen 5 7520U",
13     "vendor": "AuthenticAMD",
14     "architecture": "x86_64",
15     "topology": { "vcpus": 2 },
16     "frequency": { "max_mhz": { "value": 2794.548 } }
17 },
18 "memory": {
19     "total": { "bytes": 2013147136, "gb": 1.87 }
20 },
21 "disk": {
22     "total_disks": 1,
23     "disks": [...]
24 }
25 }
26

```

Resposta:

```

1 {
2   "message": "discovery recebido",
3   "is_virtualized": true
4 }
5

```

Código HTTP: 201 Created

POST /api/metrics

Endpoint mock para recebimento de métricas contínuas do agente. O contrato JSON com a equipe do agente ainda não foi definido — quando for, este endpoint será atualizado para validar e persistir os dados.

Payload: qualquer JSON (não validado nesta etapa)

Resposta:

```

1 {
2   "message": "metrics recebido (mock)"
3 }
4

```

Código HTTP: 201 Created

Arquivo .env.example

Criado na raiz do `BackEnd/`. Serve como modelo para que qualquer membro da equipe configure seu ambiente local sem expor credenciais reais no repositório. O arquivo `.env` real **não deve ser commitado**.

```

1 # URL de conexão com o banco PostgreSQL
2 # Será configurada junto com a equipe de banco (Beatriz) na Semana 2

```

```
3 DATABASE_URL=postgresql://app_user:senha@database:5432/monitor_db
4
5 # Modo de execução do Flask
6 # development → logs detalhados, modo debug
7 # production → modo otimizado
8 FLASK_ENV=development
9
10 # Token de autenticação da API
11 # Será aplicado nos endpoints a partir da Semana 7
12 API_KEY=troque_por_um_token_seguro
13
```

Observações técnicas

- A rota de teste `/api/data` foi removida (mantida comentada no código para histórico).
- O SQLAlchemy está inicializado mas sem modelos — nenhuma tabela é criada pelo backend. As tabelas são responsabilidade da equipe de banco de dados (Beatriz).
- O endpoint `/api/metrics` permanece como mock até que o contrato JSON seja definido com a equipe do agente (Lucas e Caio).

Pendências para a próxima semana

- Receber o contrato JSON de métricas contínuas da equipe do agente
- Criar os modelos SQLAlchemy (`Host` e `HostDiscovery`) refletindo as tabelas já criadas pela Beatriz
- Atualizar o endpoint `POST /api/discovery` para persistir os dados no banco
- @Douglas Silva @Fernando Bach - Atualizar o formato do JSON para o formato atual (no whatsapp)
- Fala nessas documentações sobre a adoção de ambientes virtuais durante o processo
- Mesmo que seja apenas na semana 2 ou 3, criar pastas de routes e models
- usar [GitHub - zluuuck/hackathon_startup](https://github.com/zluuuck/hackathon_startup) como base para construção do backend (podem literal dar git clone e copiar a estrutura)
 - OU, melhor, pesquisar como profissionais estruturam um backend em flask, usando REST API.